SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Operating Systems **COURSE CODE: CSA0404**

COURSE OUTCOME COVERED

CO4: *Optimize various file allocation strategies and disk scheduling algorithms to identify optimal methods for enhancing system performance.*

Assessment Tool Weightage: 30%

Dr VIMAL V R

QUESTIONS: 5 **Total Marks:25**

Annexure A

COMPARATIVE ANALYSIS

Code of Conduct:

I R. Indhu (192524332) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
R. Indhu
Signature of the student:

Annexure A

COMPARATIVE ANALYSIS

S.No	Comparative Analysis Questions	Marks
1	Compare **Contiguous and Linked file allocation methods** in terms of access speed, memory utilization, and fragmentation. Identify which method is more suitable for sequential file access and justify your answer.	5
2	Differentiate between **Linked and Indexed file allocation** with respect to random access capability, storage overhead, and reliability. Explain which method is more efficient for large files.	5
3	Compare **FCFS and SSTF disk scheduling algorithms** based on seek time, fairness, andthepossibilityofstarvation.Discusswhichalgorithmperformsbetterunderheavy disk load.	5
4	AnalyzethedifferencesbetweenSCANandC-SCANdiskschedulingalgorithmsin terms of head movement, response time, and uniformity of service. Which algorithm Is more suitable for real-time systems and why?	5
5	Compare **LOOK and C-LOOK disk scheduling algorithms** with respect to efficiency, total head movement, and practical implementation. Explain which algorithm provides better performance and under what conditions.	5

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

1. CONTIGUOUS VS LINKED FILE ALLOCATION

Introduction

File allocation determines how files are stored in disk blocks. Two important methods are **Contiguous Allocation** and **Linked Allocation**.

Contiguous Allocation

All blocks of a file are stored continuously.

Example:

File A → 10 → 11 → 12 → 13 → 14

Advantages

- Very fast access.
- Excellent sequential access.
- Supports fast random access.
- Less pointer overhead.
- Less disk-head movement.

Disadvantages

- External fragmentation may occur.
- File growth is difficult.
- Requires continuous free space.

Linked Allocation

File blocks can be stored anywhere. Each block contains a pointer to the next block.

10 → 25 → 7 → 42 → 18

Advantages

- No external fragmentation.
- Files can grow easily.
- Any free block can be used.
- Better storage utilization.

Disadvantages

- Random access is slow.
- Pointer requires extra space.
- Blocks may be scattered.
- More disk-head movement can occur.

Comparison

Feature	Contiguous	Linked
Access Speed	Very High	Moderate
Sequential Access	Excellent	Good
Random Access	Excellent	Poor
Fragmentation	Possible	Very Low
File Growth	Difficult	Easy
Pointer Overhead	Low	High

Conclusion

Contiguous allocation is better for sequential file access because consecutive blocks provide faster reading and reduced disk movement. Linked allocation is better when files frequently grow.

2. LINKED VS INDEXED FILE ALLOCATION

Introduction

Linked and indexed allocation allow file blocks to be stored at different disk locations. The main difference is how the blocks are located.

Linked Allocation

Each block contains the address of the next block.

10 → 20 → 35 → 50 → 75

To reach a particular block, the operating system may need to follow several pointers.

Indexed Allocation

A separate index block contains addresses of all file blocks.

```
      Index Block
    /  |  |  |  \
   10  20  35  50   75
```

The required block can be located through the index.

Comparison

Feature	Linked	Indexed
Random Access	Poor	Excellent
Sequential Access	Good	Good
Storage Overhead	Pointer per block	Index block
File Growth	Easy	Easy
Reliability	Pointer dependent	Better organized

Large Files	Good	Very Good

Random Access

Linked allocation requires following the chain:

Block 1 → Block 2 → Block 3 → Block 4

Indexed allocation can directly use the index:

Index[4] → Physical Block

Large Files

Indexed allocation is more efficient for large files, especially when random access is required.

Conclusion

Indexed allocation provides better random access and is generally more suitable for large files. Linked allocation is simpler and useful when flexible file growth is important.

3. FCFS VS SSTF DISK SCHEDULING

Introduction

Disk scheduling determines the order in which disk requests are processed. The main goal is to reduce seek time and improve performance.

FCFS

FCFS means **First Come First Serve**. Requests are processed according to their arrival order.

Example:

Head = 50

Requests:

82, 170, 43, 140, 24

Sequence:

50 → 82 → 170 → 43 → 140 → 24

Advantages

- Simple.
- Easy to implement.
- Fair.
- No starvation.

Disadvantages

- High seek time.
- Large head movements.
- Poor performance under heavy load.

SSTF

SSTF means **Shortest Seek Time First**. It selects the request closest to the current head.

Example:

Head = 50

Closest request = 43

So:

50 → 43

The algorithm continues selecting the nearest request.

Advantages

- Lower average seek time.
- Less unnecessary movement.
- Better performance under heavy load.

Disadvantages

- Starvation may occur.
- Requests far from the head may wait longer.

Comparison

Feature	FCFS	SSTF
Seek Time	High	Low
Fairness	Excellent	Moderate
Starvation	No	Possible
Complexity	Simple	Moderate
Heavy Load	Poor	Better

Conclusion

SSTF performs better under heavy disk load because it selects nearby requests and reduces average seek time. However, FCFS is fairer and does not cause starvation.

4. SCAN VS C-SCAN

Introduction

SCAN and C-SCAN are disk scheduling algorithms designed to reduce unnecessary disk-head movement and improve fairness.

SCAN

SCAN is also called the **Elevator Algorithm**.

The head moves in one direction and services requests. After reaching the end, it reverses direction.

0 ---------- 199

→

50 → 80 → 120 → 160

←

Advantages

- Good fairness.
- Reduced starvation.
- Suitable for heavy workloads.
- Better than FCFS.

Disadvantages

- Head may travel to the disk boundary.
- Waiting time is not completely uniform.

C-SCAN

C-SCAN means **Circular SCAN**.

The head services requests in one direction. After reaching the end, it returns to the beginning and continues in the same direction.

0 ---------- 199

↑ ↑

|---------- Return ----------|

Advantages

- More uniform waiting time.
- Good fairness.
- Predictable service.
- Suitable for large workloads.

Disadvantages

- Additional head movement.
- Return movement does not service requests.

Comparison

Feature	**SCAN**	**C-SCAN**
Direction	Both	One

Head Reversal	Yes	No
Response Time	Good	More Uniform
Fairness	Good	Very Good
Predictability	Good	Excellent
Real-Time Suitability	Good	Better

Real-Time Systems

Real-time systems require predictable response times. C-SCAN provides more uniform service because requests are handled in a circular direction.

Conclusion

C-SCAN is more suitable for real-time systems where predictable and uniform disk access is important.

5. LOOK VS C-LOOK

Introduction

LOOK and C-LOOK are improved versions of SCAN and C-SCAN. They avoid unnecessary movement to the physical ends of the disk.

LOOK

LOOK moves in one direction and services requests. Instead of going to the disk boundary, it reverses when there are no more requests in that direction.

Example:

Requests:

20, 50, 80, 120, 160

Head = 80

80 → 120 → 160

↓

50 → 20

Advantages

- Less head movement.
- Faster than SCAN in many cases.
- Does not travel unnecessarily to disk boundaries.
- Efficient implementation.

C-LOOK

C-LOOK works like C-SCAN but does not travel to the disk boundary.

Example:

80 → 120 → 160

↓

20 → 50

After reaching the last request, it jumps to the first pending request and continues in the same direction.

Advantages

- Less head movement.
- Uniform service.
- No unnecessary boundary travel.
- Suitable for large request queues.

Comparison

Feature	**LOOK**	**C-LOOK**
Based On	SCAN	C-SCAN
Direction	Both	One
Disk Boundary Travel	No	No
Head Reversal	Yes	No
Efficiency	Very High	Very High
Service Uniformity	Good	Very Good
Implementation	Moderate	Moderate

When to Use LOOK

LOOK is useful when:

- Low head movement is required.
- Requests are concentrated in different areas.
- Bidirectional servicing is useful.

When to Use C-LOOK

C-LOOK is useful when:

- Uniform service is important.
- Requests are distributed across the disk.
- Predictable circular servicing is required.

Conclusion

Both LOOK and C-LOOK are efficient because they avoid unnecessary movement to disk boundaries. **LOOK is better for flexible bidirectional servicing, while C-LOOK is better when uniform service is required.**

OVERALL COMPARISON

Technique	**Main Strength**	**Main Limitation**
Contiguous	Fast access	Fragmentation
Linked	Easy file growth	Poor random access
Indexed	Fast random access	Index overhead
FCFS	Simple and fair	High seek time
SSTF	Low seek time	Starvation
SCAN	Good fairness	Boundary movement
C-SCAN	Uniform service	Extra movement
LOOK	Efficient movement	Less uniform
C-LOOK	Efficient + uniform	Slightly complex